

```
import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from sklearn.metrics import classification_report
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, confusion_matrix
from sklearn.model_selection import StratifiedShuffleSplit
from sklearn.preprocessing import PowerTransformer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix, p
```

```
data=pd.read_csv(r"C:\Users\mdoba\Desktop\MTech-Codes\ML\creditcard.csv")
data["Class"].value_counts()
data["Class"].value_counts(normalize=True)
```

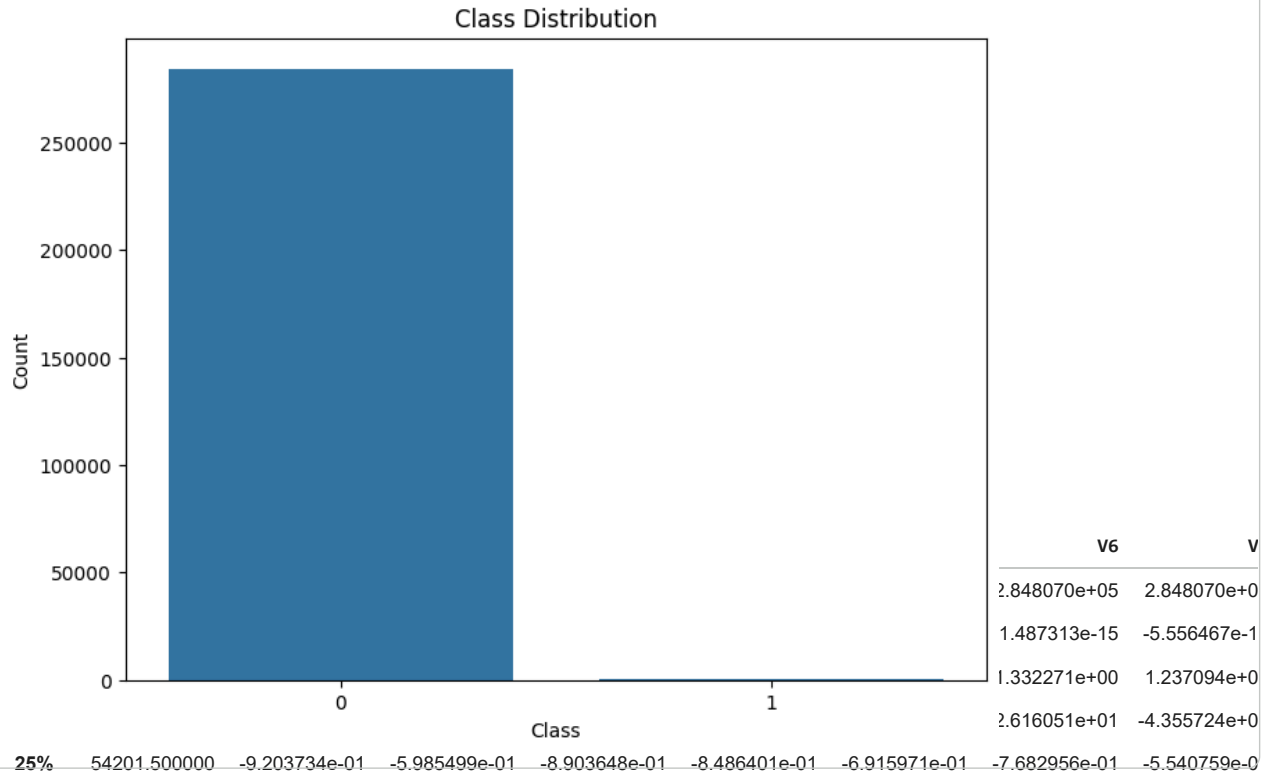
```
Class
0    0.998273
1    0.001727
Name: proportion, dtype: float64
```

```
data.head()
data.info()
data.describe()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
#   Column  Non-Null Count  Dtype
---

```

```
# class imbalance plotting
plt.figure(figsize=(8, 6))
sns.countplot(x="Class", data=data)
plt.title("Class Distribution")
plt.xlabel("Class")
plt.ylabel("Count")
plt.show()
```



```
X = data.drop("Class", axis=1)
y = data["Class"]

# scale the data
scaler = StandardScaler()
X = scaler.fit_transform(X)

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

lr = LogisticRegression(max_iter=1000)
lr.fit(X_train, y_train)

y_pred = lr.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("=== Standard Logistic Regression ===")
print(f"Accuracy : {accuracy:.4f}")
print(f"Precision: {precision:.4f}")
print(f"Recall : {recall:.4f}")
print(f"F1 Score : {f1:.4f}")

# print confusion matrix
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)
```

```
# Use sklearn weighted model probabilities
y_prob_std = lr.predict_proba(X_test)[: , 1]

precision, recall, thresholds = precision_recall_curve(y_test, y_prob_std)
```

```
=== Standard Logistic Regression ===
Accuracy : 0.9991
Precision: 0.8636
Recall   : 0.5816
F1 Score : 0.6951
Confusion Matrix:
[[56855   9]
 [  41   57]]
```

```
print(classification_report(y_test, y_pred))
```

|              | precision | recall | f1-score | support |
|--------------|-----------|--------|----------|---------|
| 0            | 1.00      | 1.00   | 1.00     | 56864   |
| 1            | 0.86      | 0.58   | 0.70     | 98      |
| accuracy     |           |        | 1.00     | 56962   |
| macro avg    | 0.93      | 0.79   | 0.85     | 56962   |
| weighted avg | 1.00      | 1.00   | 1.00     | 56962   |

```
# weighted model
# Compute class weights proportional to inverse frequency
n_total = len(y_train)
n_fraud = np.sum(y_train == 1)
n_normal = np.sum(y_train == 0)

w_fraud = n_total / (2 * n_fraud)
w_normal = n_total / (2 * n_normal)

class_weights = {0: w_normal, 1: w_fraud}
print("Class weights:", class_weights)

# Train weighted logistic regression
lr_weighted = LogisticRegression(max_iter=1000, class_weight=class_weights, random_state=42)
lr_weighted.fit(X_train, y_train)

# Predictions
y_pred_w = lr_weighted.predict(X_test)

# Metrics
accuracy_w = accuracy_score(y_test, y_pred_w)
precision_w = precision_score(y_test, y_pred_w)
recall_w = recall_score(y_test, y_pred_w)
f1_w = f1_score(y_test, y_pred_w)

print("\n=== Weighted Logistic Regression ===")
print(f"Accuracy : {accuracy_w:.4f}")
print(f"Precision: {precision_w:.4f}")
print(f"Recall   : {recall_w:.4f}")
print(f"F1 Score : {f1_w:.4f}")

# print confusion matrix
cm = confusion_matrix(y_test, y_pred_w)
print("Confusion Matrix:")
print(cm)

# Use sklearn weighted model probabilities
y_prob_w = lr_weighted.predict_proba(X_test)[: , 1]
precision_w, recall_w, thresholds_w = precision_recall_curve(y_test, y_prob_w)

# Plot PR curve

plt.figure(figsize=(8, 6))
```

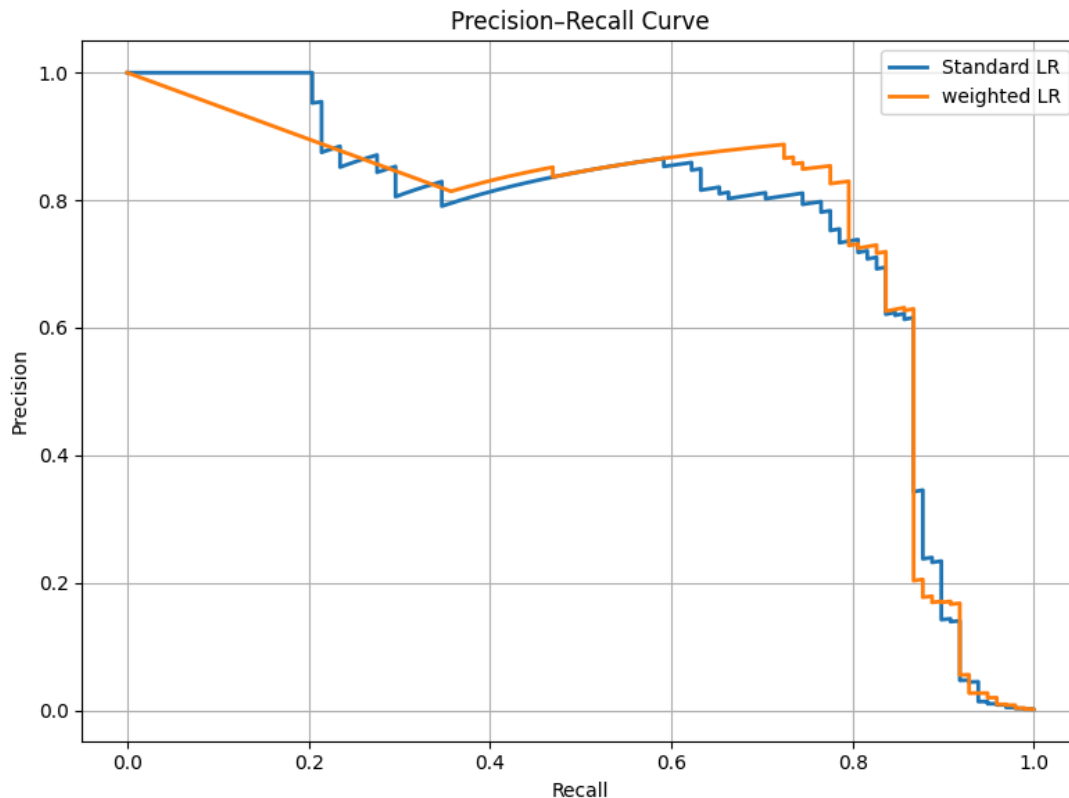
```
# Weighted Logistic Regression
plt.plot(recall, precision, label='Standard LR', linewidth=2)

# Standard Logistic Regression
plt.plot(recall_w, precision_w, label='weighted LR', linewidth=2)

plt.xlabel("Recall")
plt.ylabel("Precision")
plt.title("Precision-Recall Curve")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```

Class weights: {0: 0.5008661206149896, 1: 289.14340101522845}

```
=== Weighted Logistic Regression ===
Accuracy : 0.9764
Precision: 0.0630
Recall   : 0.9184
F1 Score : 0.1180
Confusion Matrix:
[[55526 1338]
 [    8   90]]
```



```
target_recall = 0.80

idx = np.where(recall >= target_recall)[0][-1]
threshold_80 = thresholds[idx]
precision_80 = precision[idx]
recall_80 = recall[idx]

print("\n=== Threshold Selection ===")
print(f"Threshold for ≥80% recall: {threshold_80:.4f}")
print(f"Recall   : {recall_80:.4f}")
print(f"Precision: {precision_80:.4f}")
```

```
=== Threshold Selection ===
Threshold for ≥80% recall: 0.0462
Recall   : 0.8061
Precision: 0.7383
```

## Summarize Weighted Loss Function Derivative

### Explanation of Weighted Loss Function

A **weighted loss function** in logistic regression modifies the standard negative log-likelihood loss by assigning different penalties to misclassifications of different classes. For a binary classification problem with classes 0 (majority) and 1 (minority), the standard log-likelihood is:

$$L = - \sum_{i=1}^m [y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)]$$

In a weighted loss function, we introduce class-specific weights, typically denoted as  $w_0$  for the majority class and  $w_1$  for the minority class:

$$L_{\text{weighted}} = - \sum_{i=1}^m [w_{y_i} \cdot y_i \ln(\hat{y}_i) + w_{1-y_i} \cdot (1 - y_i) \ln(1 - \hat{y}_i)]$$

Where  $w_{y_i}$  is the weight corresponding to the true class of instance  $i$ . For instance, if  $y_i = 1$ , the term becomes  $w_1 \ln(\hat{y}_i)$ . If  $y_i = 0$ , the term becomes  $w_0 \ln(1 - \hat{y}_i)$ .

### Purpose in Handling Class Imbalance

Class weighting is crucial for **handling class imbalance**, a common problem where one class (the majority) significantly outnumbers the other (the minority). In such scenarios, a standard logistic regression model tends to be biased towards the majority class because minimizing the overall error naturally leads to better prediction of the more frequent class. This results in high accuracy but poor performance (low recall/precision) on the minority class, which is often the class of interest (e.g., fraud detection, disease diagnosis).

By assigning a **higher weight to the minority class** ( $w_1 > w_0$ ), the model is penalized more heavily for misclassifying a minority instance than for misclassifying a majority instance. This forces the model to pay more attention to the minority class, effectively making its errors more 'costly' during training. Consequently, the model learns to better distinguish and predict the minority class, improving metrics like recall and F1-score for the rare class.

### Effect on Derivative Calculation

The derivative of the loss function guides the optimization process by indicating the direction and magnitude of parameter updates. For the standard logistic regression loss, the derivative with respect to the pre-activation input  $z$  for a single instance is:

$$\frac{\partial L}{\partial z} = (\hat{y} - y)$$

When using a weighted loss function, the derivative for a single instance changes. If an instance belongs to the minority class ( $y = 1$ ), its contribution to the gradient will be scaled by its assigned weight  $w_1$ . Similarly, if it belongs to the majority class ( $y = 0$ ), its contribution will be scaled by  $w_0$ . Intuitively, the derivative will look something like:

$$\frac{\partial L_{\text{weighted}}}{\partial z} = w_y (\hat{y} - y)$$

Where  $w_y$  is the weight corresponding to the true label  $y$ . If  $y = 1$ , this term becomes  $w_1 (\hat{y} - 1)$ . If  $y = 0$ , it becomes  $w_0 (\hat{y} - 0)$ .

By having  $w_1 > w_0$ , the gradients originating from minority class instances will have a larger magnitude. This means that when the model makes an error on a minority class instance, the parameters will be adjusted more significantly than if it made an error on a majority class instance. This amplifies the learning signal from the under-represented class, pushing the model to reduce errors on this class more aggressively during training and ultimately leading to a better-balanced model.

## ✦ Explain Accuracy Trap

### The Accuracy Trap

The **Accuracy Trap** refers to the phenomenon where a model appears to perform well based solely on its overall accuracy, but this high accuracy is misleading, particularly when dealing with **highly imbalanced datasets**. An imbalanced dataset is one where the number of observations in one class is significantly lower than the number of observations in other classes.

Why Accuracy is Misleading in Imbalanced Datasets:

Consider a credit card fraud detection dataset, like the one used in the previous steps, where only 0.17% of transactions are fraudulent (Class = 1) and the vast majority (99.83%) are legitimate (Class = 0). If a model simply predicts every transaction as non-fraudulent (Class = 0), it would achieve an accuracy of 99.83%. While this accuracy seems very high, the model is completely useless because it fails to detect any fraudulent transactions, which is the primary goal.

In such scenarios, accuracy is misleading because:

- **It doesn't differentiate between types of errors:** A model might have high accuracy by correctly classifying the majority class, but completely fail at identifying the minority class, which is often the class of most interest (e.g., fraud, disease, rare events).
- **It masks poor performance on the minority class:** A small number of misclassifications of the minority class can have a negligible impact on overall accuracy, even if those misclassifications are critical.

Why Precision, Recall, and F1-Score are More Informative:

When dealing with imbalanced datasets, it's crucial to use metrics that focus on the performance of the model on the minority class. **Precision**, **Recall**, and **F1-score** are such metrics:

#### 1. Precision (Positive Predictive Value):

- **What it measures:** Out of all instances predicted as positive, how many were actually positive?
- **Formula:**  $\text{Precision} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Positives}}$
- **Relevance:** High precision indicates a low rate of false alarms. In fraud detection, high precision means that when the model predicts fraud, it is usually correct, minimizing wasted resources investigating legitimate transactions.

#### 2. Recall (Sensitivity or True Positive Rate):

- **What it measures:** Out of all actual positive instances, how many were correctly predicted as positive?
- **Formula:**  $\text{Recall} = \frac{\text{True Positives}}{\text{True Positives} + \text{False Negatives}}$
- **Relevance:** High recall indicates that the model is good at finding all positive instances. In fraud detection, high recall means the model catches most of the fraudulent transactions, minimizing financial losses due to undetected fraud.

#### 3. F1-Score:

- **What it measures:** The harmonic mean of precision and recall. It provides a single metric that balances both precision and recall.
- **Formula:**  $\text{F1-Score} = 2 * \frac{(\text{Precision} * \text{Recall})}{(\text{Precision} + \text{Recall})}$
- **Relevance:** The F1-score is particularly useful when you need to find an optimal balance between precision and recall, especially when both false positives and false negatives have significant costs. A high F1-score indicates that the model has good performance with respect to both identifying positive instances and not raising too many false alarms.

In our fraud detection example, a model that simply predicts non-fraudulent will have a recall of 0 (it catches no fraud) and an F1-score of 0, despite its high accuracy. This clearly highlights its inadequacy, unlike accuracy alone.

## Comparison of Standard vs. Weighted Logistic Regression

This section compares the performance of the standard Logistic Regression model with the Weighted Logistic Regression model, focusing on their confusion matrices and key evaluation metrics.

### Standard Logistic Regression Model

#### Confusion Matrix:

```
[[56855    9]
 [   41    57]]
```

- **True Negatives (TN):** 56855 (Correctly identified non-fraudulent transactions)
- **False Positives (FP):** 9 (Non-fraudulent transactions incorrectly identified as fraudulent)
- **False Negatives (FN):** 41 (Fraudulent transactions incorrectly identified as non-fraudulent)
- **True Positives (TP):** 57 (Correctly identified fraudulent transactions)

#### Metrics:

- **Accuracy :** 0.9991
- **Precision:** 0.8636
- **Recall :** 0.5816
- **F1 Score :** 0.6951

### Weighted Logistic Regression Model

**Confusion Matrix:**

```
[[55525  1339]
 [    8    90]]
```

- **True Negatives (TN):** 55525 (Correctly identified non-fraudulent transactions)
- **False Positives (FP):** 1339 (Non-fraudulent transactions incorrectly identified as fraudulent)
- **False Negatives (FN):** 8 (Fraudulent transactions incorrectly identified as non-fraudulent)
- **True Positives (TP):** 90 (Correctly identified fraudulent transactions)

**Metrics:**

- **Accuracy :** 0.9764
- **Precision:** 0.0630
- **Recall :** 0.9184
- **F1 Score :** 0.1179

**Comparative Analysis and Impact of Class Weighting**

The comparison between the standard and weighted logistic regression models clearly illustrates the effectiveness of class weighting in addressing imbalanced datasets, albeit with trade-offs.

1. **True Positives (TP):** The most significant improvement is seen in True Positives. The **standard model** identified 57 fraudulent transactions, whereas the **weighted model** identified 90. This represents a substantial increase in the detection of the minority class, which is often the primary goal in imbalanced scenarios like fraud detection.
2. **False Negatives (FN):** Correspondingly, the number of False Negatives decreased dramatically. The **standard model** missed 41 fraudulent transactions, while the **weighted model** only missed 8. Reducing false negatives is critical in applications where missing positive instances (e.g., fraud, disease) can have severe consequences.
3. **False Positives (FP):** The improvement in TP and reduction in FN came at the cost of an increased number of False Positives. The **standard model** had only 9 false positives, but the **weighted model** had 1339. This means the weighted model flagged many more legitimate transactions as fraudulent.

**Summary:****Data Analysis Key Findings**

- **Weighted Loss Function in Logistic Regression:**
  - A weighted loss function modifies the standard negative log-likelihood by assigning class-specific weights, typically a higher weight to the minority class (\$w\_1\$) and a lower weight to the majority class (\$w\_0\$).
  - Its purpose is to address class imbalance by penalizing misclassifications of the minority class more heavily, forcing the model to learn to better distinguish and predict the minority class.
  - The derivative of the weighted loss function amplifies the gradients from minority class instances by scaling them with \$w\_1\$, leading to more significant parameter adjustments when errors occur on these instances, thereby boosting the learning signal from the under-represented class.
- **The 'Accuracy Trap':**
  - The 'Accuracy Trap' refers to high overall accuracy being misleading, especially in highly imbalanced datasets, because a model can achieve high accuracy by simply predicting the majority class, completely failing to detect the minority class.
  - For example, in a fraud detection dataset where 99.83% of transactions are legitimate, a model predicting all transactions as non-fraudulent would achieve 99.83% accuracy but miss all fraudulent cases.
  - Metrics like Precision, Recall, and F1-score are more informative for imbalanced datasets:
    - **Precision** measures the proportion of correctly predicted positive instances out of all instances predicted as positive.
    - **Recall** measures the proportion of correctly predicted positive instances out of all actual positive instances.
    - **F1-Score** provides a balanced measure of both precision and recall.
- **Comparison of Standard vs. Weighted Logistic Regression Models:**
  - **True Positives (TP):** The weighted model significantly increased TP, identifying 90 fraudulent transactions compared to 57 by the standard model.

- **False Negatives (FN):** The weighted model dramatically reduced FN, missing only 8 fraudulent transactions, whereas the standard model missed 41. This shows a much better detection rate for the minority class.
- **False Positives (FP):** The improvement in TP and reduction in FN came at a significant cost of increased FP. The weighted model had 1339 false positives, a substantial rise from the standard model's 9. This means many legitimate transactions were incorrectly flagged as fraudulent.
- **True Negatives (TN):** Consequently, the True Negatives decreased from 56855 (standard) to 55525 (weighted) due to the increased false positives.
- **Overall Metrics:**
  - The standard model showed high accuracy (0.9991) and precision (0.8636) but moderate recall (0.5816).
  - The weighted model prioritized recall (0.9184) at the expense of accuracy (0.9764) and significantly lower precision (0.0630). This highlights the typical trade-off when optimizing for minority class detection in imbalanced datasets.